

Logic Design HW#4

蔡定宇 112006265 (IBP EECS)

Source code of Verilog Design:

```
//filename signed_ripple_adder.v
```

```
// Description of half adder
```

```
module halfadder (S,C,x,y);
```

```
    input x,y;
```

```
    output S,C;
```

```
    //Instantiate primitive gates
```

```
    xor (S,x,y);
```

```
    and (C,x,y);
```

```
endmodule
```

```
//Description of full adder
```

```
module Fulladder (S,C,x,y,z);
```

```
    input x,y,z;
```

```
    output S,C;
```

```
    wire S1,D1,D2;
```

```
    //Instantiate the halfadder
```

```
    halfadder    HA1 (S1,D1,x,y),
```

```
                HA2 (S,D2,S1,z);
```

```
    or g1(C,D2,D1);
```

```
endmodule
```

```
module AdderWithOverflow(A,B,S,E,Cin);
```

```
    input [7:0] A, B;
```

```
    output [7:0] S;
```

```
output E;

    input Cin;

wire [7:1] c;

wire [7:0] sum;

wire carry;
```

```
Fulladder    FA0 (S[0], c[1], Cin, A[0], B[0]),

               FA1 (S[1], c[2], c[1], A[1], B[1]),
               FA2 (S[2], c[3], c[2], A[2], B[2]),
               FA3 (S[3], c[4], c[3], A[3], B[3]),
               FA4 (S[4], c[5], c[4], A[4], B[4]),
               FA5 (S[5], c[6], c[5], A[5], B[5]),
               FA6 (S[6], c[7], c[6], A[6], B[6]),
               FA7 (S[7], carry, c[7], A[7], B[7]);
```

```
xor(E,carry,c[7]);
```

```
endmodule
```

Source code of Verilog testbench:

```
//filename tb_signed_ripple_adder.v
```

```
//filename tb_signed_ripple_adder.v
```

```
`timescale 1ns/1ps
```

```
module testbench;
```

```
    reg signed [7:0] A, B;
```

```
        reg Cin;
```

```
    wire signed [7:0] S;
```

```
    wire E;
```

```
AdderWithOverflow UUT (A,B,S,E,Cin);
```

```
initial begin
```

```
    $dumpfile("wave.vcd");
```

```
    $dumpvars(0, testbench);
```

```
end
```

```
initial begin
```

```
    // Test case 1
```

```
        Cin = 0;
```

```
        A = 8'sb000000101; // 5
```

```
        B = 8'sb000001010; // 10
```

```
        #10; // Wait for a few simulation cycles
```

```
        $display("Test Case 1: A = %d, B = %d, S = %d, E = %d", A, B, S, E);
```

```
    // Test case 2
```

```
        Cin = 0;
```

```
        A = 8'sb01010111; // 87
```

```
        B = 8'sb11101001; // -23 (2's complement)
```

```
        #10; // Wait for a few simulation cycles
```

```
        $display("Test Case 2: A = %d, B = %d, S = %d, E = %d", A, B, S, E);
```

```
    // Test case 3
```

```
        Cin = 0;
```

```
        A = 8'sb10001000; // -120 (2's complement)
```

```
        B = 8'sb00111100; // 60
```

```
        #10;
```

```
$display("Test Case 3: A = %d, B = %d, S = %d, E = %d", A, B, S, E);
```

```
// Test case 4
```

```
    Cin = 0;
```

```
A = 8'sb10110000; // -80 (2's complement)
```

```
B = 8'sb11000000; // -64 (2's complement)
```

```
#10;
```

```
    $display("Test Case 4: A = %d, B = %d, S = %d, E = %d", A, B, S, E);
```

```
    Cin = 0;
```

```
A = 8'sb10000101;
```

```
B = 8'sb10010100;
```

```
#10;
```

```
    $display("Test Case 5: A = %d, B = %d, S = %d, E = %d", A, B, S, E);
```

```
    Cin = 0;
```

```
A = 8'sb01010011;
```

```
B = 8'sb00001111;
```

```
#10;
```

```
$display("Test Case 6: A = %d, B = %d, S = %d, E = %d", A, B, S, E);
```

```
$finish;
```

```
end
```

```
endmodule
```

Simulation results:

